

Alert Triage Workflow

Problem statement :

Security engineers working with cloud environments often receive a large number of security alerts everyday. These alerts may vary in severity and may require different levels of investigation. When alerts are presented without a clear priority, relevant context, or an easy way to take action, engineers may spend unnecessary time investigating low-priority alerts while important threats require immediate attention.

Alert Triage Workflow , it is a way for a security engineer to see, investigate and resolve a cloud security alert in one connected flow.

User Persona:

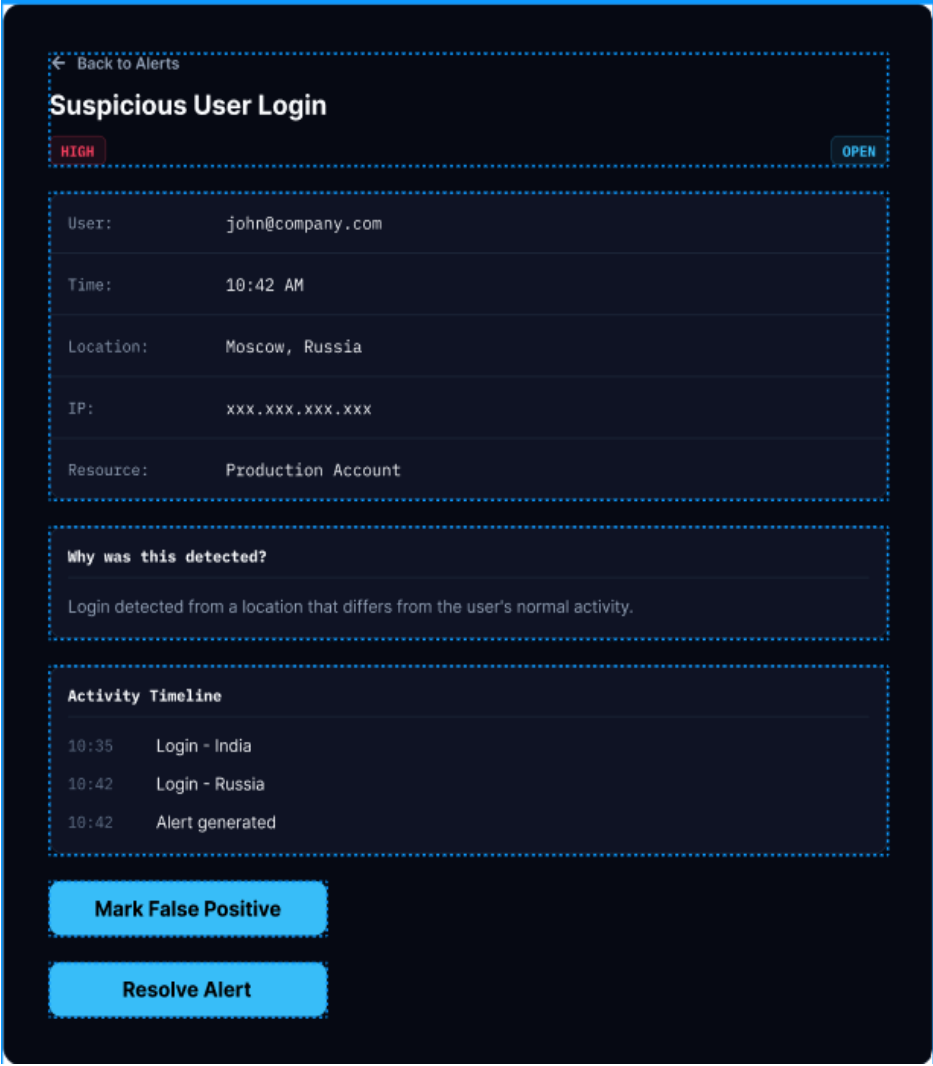
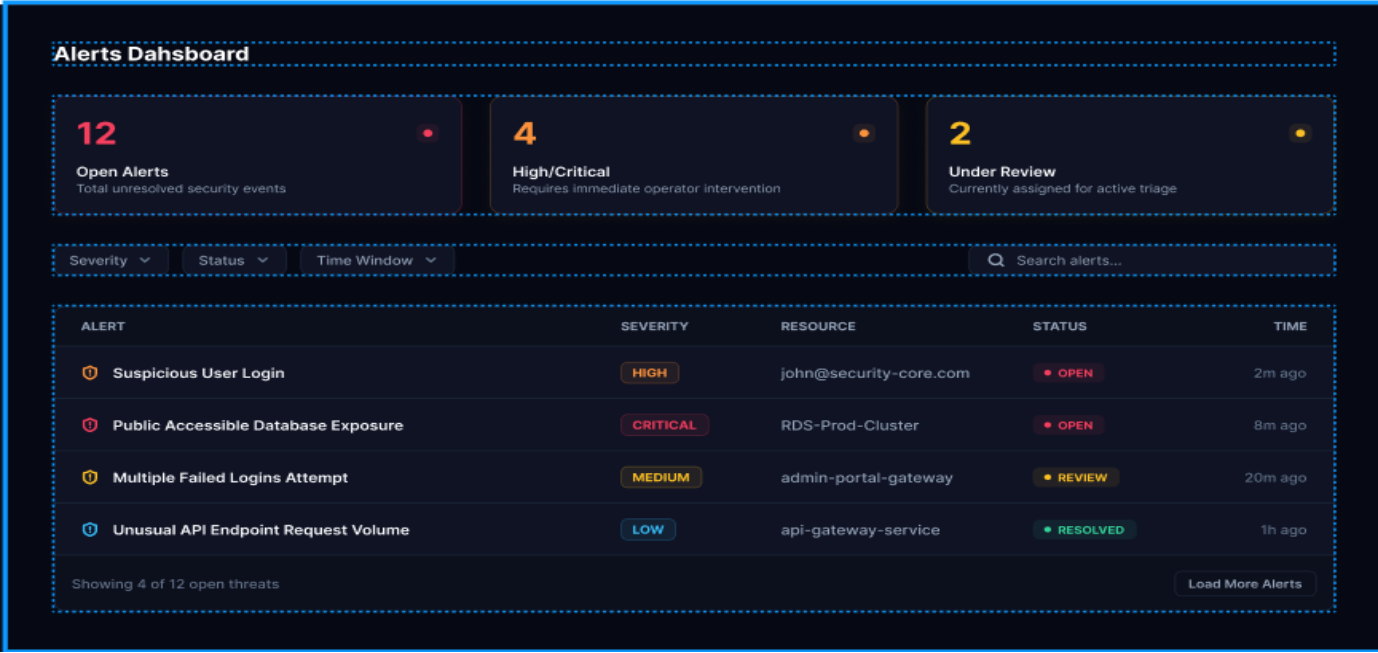
The primary user here is a security engineer who is responsible for monitoring and responding to security alerts in an organization's cloud environment.

Goals:

- Identify the most important alerts quickly.
- Understand why an alert was generated.
- Investigate suspicious activity efficiently.
- Resolve genuine incidents quickly and avoid wasting time on false positives.
- Maintain a record of how alerts were handled.

Pain Points :

- Alert fatigue — high volume of low-value or duplicate alerts buries the critical ones.
- Context switching — has to jump between the security tool, cloud console, and ticketing system to get the full picture.
- Missing business context — an alert may say “excessive permissions” but not show whether the resource is production-critical or who owns it.
- Weak audit trail — hard to show what was investigated and when, which matters for compliance.



The proposed Alert Triage Workflow is designed to help security engineers efficiently investigate and resolve cloud security alerts through a simple, focused workflow. The solution consists of **three** primary stages: **alert prioritization**, **investigation**, and **resolution**.

The features proposed are :

1. Alert List which provides visibility into incoming and existing alerts
2. Filter to filter out severity and status of alerts which can help decide which alert to investigate at the earliest
3. Alert details which provides context regarding an alert which is needed for further investigation
4. Investigation timeline which helps to understand what happened
5. Resolution reason : add notes regarding the closing of an alert
6. Search button which allows the engineer to directly search for an alert

Features are prioritized according to their importance to the core triage workflow. **Alert visibility, severity and status, alert details, investigation information, and resolution** are considered **must-have features** because they directly enable the engineer to complete the primary task.

Filtering, search, and investigation actions are considered **should-have features** because they improve efficiency when dealing with a larger number of alerts but are not required for the basic workflow.

The success of the workflow can be measured using metrics like

1. **Mean Time to Triage:** Average time required for an engineer to understand an alert and determine the appropriate action.
2. **Mean Time to Resolution:** Average time taken from opening an alert to resolving it.
3. **Alert Resolution Rate:** Percentage of assigned alerts successfully resolved.
4. **False Positive Handling Time:** Average time required to identify and close an alert as a false positive
5. **Workflow Completion Rate:** Percentage of alerts successfully taken through the complete investigation and resolution workflow.

One aspect that could be discussed with the development team is how to prevent conflicting investigations

Two security engineers might open and investigate the **same alert** simultaneously.

The development team could discuss how to handle this.

This can be implemented by :

1. Locking or reserving an alert
2. Showing who is investigating it currently
3. What happens if the engineer leaves without resolving it
4. Whether another engineer can take over